

Build an AI Trading Research Workflow with Codex and GPT 6 Astra

This guide explains the process we used: research the market, connect **Alpaca**, schedule a small number of daily checks, and send two **ClickUp** updates. The completed setup supports research, account monitoring, and reporting. Live order execution was still disabled when this guide was prepared.

By: [Nate Herk](#)

1. Research the challenge before building the bot

We gave **GPT 6 Astra** a seven market day challenge and asked it to research the market backdrop, upcoming economic releases, company earnings, and potential stock/ETF opportunities. We used web research, prioritizing official economic calendars, Federal Reserve announcements, and company investor relations pages.

Turn research into rules

We turned that research into a **Markdown strategy**: a short watchlist, reasons to consider each candidate, entry and exit conditions, position sizing, loss limits, and events to avoid. The proposed approach focused on liquid stocks/ETFs, confirmed price moves, limited entries, and finishing each day in cash. These were proposed rules, not a proven profitable strategy.

Choose your own challenge inputs

For replication, give Codex your own dates, starting budget, permitted instruments, and loss limits. Ask it to cite sources and refresh the calendar before the challenge. We settled on web research plus Alpaca prices, historical bars, and news rather than adding several paid research plugins. Better data coverage does not establish a trading edge.

2. Create a dedicated Codex project

We moved the setup into a separate project folder, selected GPT 6 Astra, and created one ongoing conversation for the challenge. This kept the strategy, credentials, schedules, and progress records together.

Ask Codex to create these project files:

- **AGENTS.md**: the operating rules and files every run must read.
- docs/TRADING-STRATEGY.md: research, watchlist, and proposed risk rules.
- state/readiness.json: what has passed verification and whether live execution is enabled.
- routines/schedule.json: session dates, timezone, check times, and report ownership.
- A local progress journal and handoff file: what happened and what the next run should do.

3. Connect Alpaca and test read access

We initially explored the Alpaca plugin, then used direct Alpaca API access from a small Python client. Installing a plugin alone did not prove that the account or data connection worked.

Credentials and account mode

To replicate, create your own Alpaca API key and secret in its dashboard. Use paper credentials for simulated trading, and explicitly select paper or live. Have Codex create a project local `.env` file with `ALPACA_API_KEY`, `ALPACA_SECRET_KEY`, and `ALPACA_ENVIRONMENT`. Enter the real values locally, exclude `.env` from Git, and keep it out of shared files and logs. Never paste credentials into the replication prompt.

We mapped three services: live brokerage at `api.alpaca.markets`, paper brokerage at `paper-api.alpaca.markets`, and market data at `data.alpaca.markets`. The client supplied credentials through Alpaca's authentication headers.

Verify the connection

We tested account balances/status, positions, open orders, the market clock, account settings, an asset lookup, and samples of fills, historical bars, news, and consolidated **SIP** quotes. All calls were read only. Codex saved the results locally and distinguished verified access from missing checks.

Check market data coverage

We selected **Algo Trader Plus** for broader market coverage. Alpaca's Basic plan includes limited real time stock data from IEX; it is not entirely historical only. A successful SIP request confirmed access to that feed, but did not prove subscription billing or fresh prices during an open session. Verify both separately. See [Alpaca market data documentation](#).

4. Set up the scheduled checks

We first tried separate schedules returning to the same conversation. The app rejected multiple active heartbeats on one task, so we combined the exact times into **one active automation**. These are Codex scheduled checks, not a separately installed operating system cron service. See [Codex scheduled tasks documentation](#).

Our ordinary session schedule used America/Chicago time:

- 7:45 a.m.: review news, reconcile the account, and prepare the plan.
- 9:30 a.m.: review the first eligible entry opportunity.
- 11:00 a.m.: review positions and the final entry opportunity.
- 1:00 p.m.: review existing positions and send the midday update.
- 2:15 p.m.: review flattening positions under the approved execution rules.
- 2:45 p.m.: verify positions/orders and reconcile results.
- 3:15 p.m.: send the after close report.

Final session adjustments

On our final session, the last three checks moved to noon, 12:30 p.m., and 3:15 p.m. to finish before a major scheduled event. The midday report moved to 12:30 p.m. This produced 48 planned checks and 14 reports across seven sessions. For another challenge, rebuild event overrides around its actual dates.

Ask Codex to attach the combined schedule to the existing challenge conversation, verify every upcoming occurrence, exclude non session dates, and stop after the final session. Keep the local computer awake and Codex running. Scheduled starts can be delayed; they cannot substitute for broker held protection.

5. Give every run a reliable handoff

Each wake up follows the same process: read instructions and readiness, load the previous handoff, check for unfinished work, record the run's start, reconcile fresh broker data, perform that slot's job, and save the outcome plus the next job.

Save progress outside the conversation

We implemented a local **SQLite journal** with generated Markdown progress logs. A duplicate or unfinished run blocks new trading activity. A separate process successfully recovered a completed read only rehearsal, demonstrating continuity beyond conversation memory. The journal records actual observations; it does not invent fills or replace the broker's records.

6. Connect ClickUp and send two daily updates

We connected ClickUp, selected a private reporting channel, and explicitly authorized messages there. To replicate, have Codex resolve your own channel and recipient, then send a test message and read it back. Use your own destination, not identifiers copied from another setup.

- Midday: market developments, trades or no trade reasons, current positions, balances, and the afternoon plan.
- After close: trades and outcomes, ending balances, remaining positions/orders, verified performance, and next steps.

Confirm delivery and prevent duplicates

Both reports send even if no trades occurred. We added the intended recipient as a follower and saved a unique date/report key plus the returned message ID to prevent duplicates. After an uncertain send, Codex checks channel history before retrying. A successful channel post does not prove that a phone notification appeared.

7. Verify the setup before enabling live orders

We verified the saved schedule, account/data reads, ClickUp delivery, and a read only handoff rehearsal. Thirteen offline connection and logging tests passed. We had not completed the order controller, trading risk ledger, paper order/protection rehearsal, or all account specific restrictions. **Live trading remained disabled.** Replicating the monitoring setup does not complete those execution steps.

A starter prompt to use in Codex

Copy into Codex: Use GPT 6 Astra to help me build a seven market day trading research and monitoring project. First ask for my dates, budget, allowed instruments, and loss limits. Research official calendars and company announcements, and save a cited Markdown strategy. Create project instructions, readiness tracking, a project local Alpaca .env template, read only account/data checks, and a durable progress journal. Keep real credentials out of chat and shared files. Configure the check times in this guide through one automation returning to this conversation, adjusted for my session dates and timezone. Connect my chosen ClickUp channel and send midday and after close reports, including no trade days. Test account reads, handoff recovery, schedule dates, and notification delivery. Leave live orders disabled until execution controls and paper trading tests have passed and I explicitly authorize execution.

Key Takeaways

1. Start with a cited strategy, defined risk limits, and a current event calendar.
 2. Keep the project, credentials, and operating instructions together.
 3. Verify Alpaca account and data access before depending on the connection.
 4. Use one combined automation to keep every scheduled check in the same conversation.
 5. Save each run's outcome and next job, and verify both daily ClickUp reports.
 6. Keep live orders disabled until execution controls and paper tests have passed.
-

Want to connect with others building and monetizing AI automation?

[**Become an AIS Plus Member**](#)